

Plan MVP — Always on Shelf (Hackathon Tuali 2026)

Equipo: Jhulyam · Emi · Carlos · Marita **Duración:** 48 horas **Reto principal:** Always on Shelf: Surtido predictivo para canal moderno **Reto secundario (Plan B):** Churn Hunters — Anticipándonos al abandono de clientes

1. El reto literal

"Desarrollar una solución de IA capaz de aprender un proceso entre sistemas web observándolo una sola vez y ejecutarlo de forma autónoma, eliminando tareas manuales, errores y tiempo operativo."

Traducción: Construir un agente que **aprende por demostración** flujos en navegador y los **ejecuta autónomamente**, generalizando a nuevos inputs.

NO es: un modelo ML de predicción de surtido. Esa es la trampa. El ángulo ganador es **learning-by-demonstration / browser agent**.

2. Equipo y roles

Persona	Rol	Responsabilidad principal
Jhulyam	Tech Lead + Agent Engineer + Arquitecto	Owner total del cerebro técnico: agent loop, Claude SDK + Computer Use, browser automation (Stagehand), lógica de generalización. Decide stack y arquitectura. Code review de todo lo técnico. Punto único de decisión técnica.
Carlos	Frontend Engineer + Co-Pitch (hora 32+)	Implementa el Operator UI siguiendo los mocks de Marita. Codea componentes, integra con el backend, conecta con el agente. Hora 32+ apoya pitch (ensayos, review de slides).
Marita	UI/UX Lead	Diseña mockups y dirección visual del Operator UI, del Mock Portal y de los slides del pitch. Diseña componentes. Polish del demo final. Apoya animaciones de UI.
Emi	Pitch Lead + Backend Support	Dueño de narrativa de 3 actos, slides, métricas de negocio, video backup, ensayos. En paralelo construye el mock portal (Next.js fake site), schemas de Supabase y datos mock para demos.

Carga balanceada:

- Jhulyam toma los 2 frentes más complejos (agent loop + browser automation) porque son los más arriesgados y requieren visión arquitectónica unificada.
- Carlos absorbe TODO el código del frontend del operador → Jhulyam no se distrae con UI.
- Emi cubre el backend "fácil" (mock portal + seeds) en paralelo a su rol de pitch, porque el pitch deck no requiere 48h de trabajo continuo.
- Marita asegura que el producto se vea premium — clave para el wow del demo.

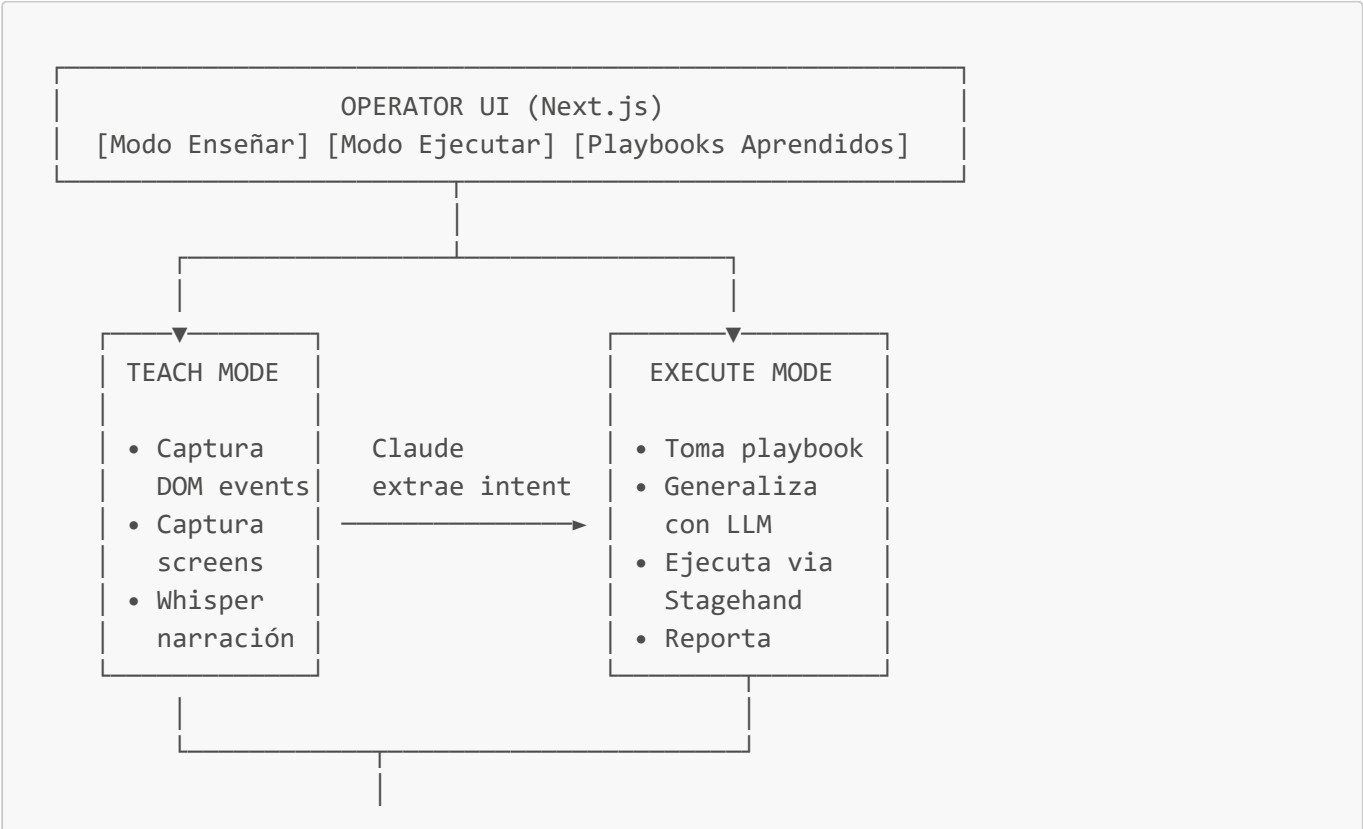
Reconfiguren si: el skill-check de 15 min al arranque revela que alguien domina otra área mejor o tiene preferencia fuerte.

3. Stack técnico

Capa	Herramienta	Por qué
Orquestación agente	@anthropic-ai/sdk con Claude Agent SDK + Computer Use beta	Estado del arte, integración nativa
Browser control	Stagehand (Browserbase)	Más confiable para demos en vivo que browser-use
Frontend (Operator UI + Mock Portal)	Next.js 15 + Tailwind + shadcn/ui	Velocidad de desarrollo, look profesional
Storage	Supabase (Postgres + Auth + Storage)	Setup en minutos, guarda playbooks + grabaciones
Audio → texto	OpenAI Whisper API	Para la narración durante el modo enseñar
Hosting	Vercel (frontend) + Browserbase (navegador headless)	Predecible para demo

Costos estimados: ~\$50-100 USD en APIs durante desarrollo + demo. **Pedir créditos a organizadores el día 0.**

4. Arquitectura





Flujo del playbook (concepto clave)

Un **playbook** es un JSON que describe el flujo grabado en términos abstractos, no en clicks exactos. Ejemplo:

```
{
  "name": "actualizar-surtido-canal-moderno",
  "intent": "Actualizar cantidad de SKU X en lista de tiendas",
  "steps": [
    { "action": "navigate", "target": "/portal/login" },
    { "action": "fill", "selector_intent": "campo de usuario", "value": "{credentials.user}" },
    { "action": "click", "selector_intent": "botón ingresar" },
    { "action": "navigate", "target": "/portal/surtido" },
    { "action": "for_each", "input": "{stores}", "as": "store", "steps": [
      { "action": "click", "selector_intent": "tienda con código {store.id}" },
      { "action": "fill", "selector_intent": "cantidad de {sku}", "value": "{quantity}" },
      { "action": "click", "selector_intent": "botón guardar" }
    ]
  },
  ],
  "parameters": ["stores", "sku", "quantity", "credentials"]
}
```

Esto es lo que da el "wow": El playbook es legible, parametrizable, y Stagehand resuelve los `selector_intent` en tiempo de ejecución con visión + DOM.

5. Timeline 48h

Día 1 (horas 0-24): Construir las piezas

Hora	Hito	Responsable
0-1	Kickoff + skill-check (15 min) + repo monorepo + .env templates + Anthropic/Browserbase keys	Todos
1-2	Decisiones de arquitectura (consultar <code>agents/architect.md</code> de ECC)	Jhulyam

Hora	Hito	Responsable
2-8	Computer Use POC: agente abre URL, hace click, escribe texto	Jhulyam
2-8	Mock portal v1: login fake, lista de tiendas, form de surtido con <code>data-testid</code> por todas partes	Emi
2-8	Operator UI scaffold + componentes base (chat, lista de playbooks, viewer)	Carlos
2-8	Mockups visuales del Operator UI + Mock Portal + dirección de diseño	Marita
2-8	Pitch deck v0 (estructura, narrativa de 3 actos, NO contenido aún)	Emi (en paralelo)
8-16	Integración 1: Modo Enseñar end-to-end → usuario hace flujo, queda <code>playbook.json</code> en Supabase	Jhulyam + Carlos
8-16	Schemas Supabase + seeds + datos mock para demo	Emi
8-16	Refinamiento de mockups + componentes visuales en Figma → Carlos los implementa	Marita
16-24	Integración 2: Modo Ejecutar → agente toma <code>playbook.json</code> y replica IDÉNTICO	Jhulyam
16-24	UI del modo ejecución (ver navegador en vivo, ver pasos siendo ejecutados)	Carlos
16-24	Polish visual del Mock Portal (que se vea como portal corporativo real)	Marita + Emi

🚦 **Checkpoint hora 24:** Si NO hay demo idéntico end-to-end → evaluar pivot al Plan B (Growth Agent). Decisión binaria, no negociable.

Día 2 (horas 24-48): Pulir y ganar

Hora	Hito	Responsable
24-32	Generalización: LLM toma playbook + nuevos inputs → ajusta valores y selectores. Esto es lo que gana el hack.	Jhulyam
24-32	Pulido del modo ejecución con animaciones (consultar <code>skills/motion-ui</code> de ECC)	Carlos + Marita
24-32	Pitch deck v1: contenido completo, métricas concretas, narrativa final	Emi
32-40	Edge cases visibles ("agente reintenta", "agente reporta éxito")	Jhulyam
32-40	UI demo polish (consultar <code>skills/ui-demo</code> , <code>make-interfaces-feel-better</code>)	Marita + Carlos
32-40	Carlos pasa de coding a co-pitch: review de slides, contraprueba de narrativa	Carlos

Hora	Hito	Responsable
32-40	Métricas de negocio finalizadas (cálculo de ahorro \$\$\$)	Emi
40-44	Grabar backup video del demo perfecto + subtítulos	Emi + Marita
44-46	3 ensayos cronometrados del pitch completo	Todos
46-48	Buffer para bugs de último minuto + descanso final	Quien lo necesite

6. Demo strategy: los 3 actos que ganan

Acto 1 — El dolor (45 seg)

"Así trabaja hoy un planeador de Arca."

Persona en cámara hace el flujo manualmente en el portal mock. Cronómetro visible: **3 min de trabajo, multiplicar por 12 procesos diarios.**

Acto 2 — La enseñanza (60 seg)

"Le mostramos al agente UNA VEZ."

Click en "Modo Enseñar", repiten el flujo narrando. Sistema graba. Aparece el playbook en la UI con pasos legibles en lenguaje humano.

Acto 3 — EL MOMENTO GANADOR (90 seg)

"Ahora le pedimos lo mismo para 5 tiendas distintas con un SKU diferente."

Input rápido en la UI. Click en "Ejecutar". El navegador se mueve solo en pantalla grande. Termina en ~20 segundos. **Silencio del jurado.**

Cierre — La métrica (30 seg)

Slide con números:

- "45 min/proceso × 12 procesos/semana × 8 planeadores = **72 horas/semana eliminadas**"
- "~\$XXX,XXX/año ahorrados solo en este flujo"
- "Y aprende cualquier proceso entre sistemas web, no solo este"

Total: 3:45 min — dentro del límite típico de pitch.

7. ECC: cherry-picks de alto valor

ECC ya está clonado en [exoesqueleto/ECC/](#). **No instalen el framework completo** — úsenlo como **biblioteca de referencia**. Cuando alguien vaya a hacer X, lee el archivo correspondiente antes.

Skills DIRECTAMENTE relevantes (lectura obligatoria por rol)

Quién	Cuándo	Qué leer
Jhulyam	Hora 0-2	exoesqueleto/ECC/skills/agent-harness-construction/SKILL.md
Jhulyam	Hora 2-8	exoesqueleto/ECC/skills/autonomous-agent-harness/SKILL.md
Jhulyam	Hora 8-16	exoesqueleto/ECC/skills/click-path-audit/SKILL.md
Jhulyam	Hora 16-24	exoesqueleto/ECC/skills/browser-qa/SKILL.md
Carlos	Hora 0-2	exoesqueleto/ECC/skills/frontend-patterns/SKILL.md
Carlos	Hora 2-8	exoesqueleto/ECC/skills/react-patterns/SKILL.md
Carlos	Hora 24-32	exoesqueleto/ECC/skills/motion-ui/SKILL.md
Marita	Hora 0-8	exoesqueleto/ECC/skills/frontend-design-direction/SKILL.md
Marita	Hora 24-32	exoesqueleto/ECC/skills/motion-foundations/SKILL.md + motion-patterns/SKILL.md
Marita	Hora 32-40	exoesqueleto/ECC/skills/make-interfaces-feel-better/SKILL.md + ui-demo/SKILL.md
Emi	Hora 0-8	exoesqueleto/ECC/skills/frontend-slides/SKILL.md (template HTML listo)
Emi	Hora 2-8	exoesqueleto/ECC/skills/investor-materials/SKILL.md
Emi	Hora 24-32	exoesqueleto/ECC/skills/ui-demo/SKILL.md

Subagentes útiles (referencia en momentos puntuales)

Cuándo	Cuál	Para qué
Hora 1-2	exoesqueleto/ECC/agents/architect.md	Decidir estructura del repo y stack final
Hora 8+, antes de merges	exoesqueleto/ECC/agents/typescript-reviewer.md	Review rápido de código TS
Hora 8+, antes de merges	exoesqueleto/ECC/agents/react-reviewer.md	Review de componentes Next.js
Hora 32+	exoesqueleto/ECC/agents/silent-failure-hunter.md	Cazar bugs invisibles antes del demo
Hora 40+	exoesqueleto/ECC/agents/security-reviewer.md	Scan rápido pre-demo

Lo que NO usamos

- Memoria persistente del ECC (overhead innecesario para 48h)
- Hooks (riesgo de debugging del framework)
- 200+ skills restantes (ruido)
- `install.ps1` / `install.sh` completo (no necesitamos config global)

8. Riesgos y mitigaciones

Riesgo	Probabilidad	Mitigación
Computer Use latente (2-5 seg/paso) hace demo lento	Alta	Grabar video del demo a 2x, mostrar speed-up explícito en pitch
Selectores frágiles en generalización	Media	Mock portal con <code>data-testid</code> por todas partes + Stagehand vision-first
API costs se disparan en pruebas	Media	Modo "playback" sin LLM para pruebas repetidas; LLM solo en runs finales
Demo en vivo falla por red / timeout	Media-Alta	Video backup obligatorio en hora 40
Jhulyam se vuelve cuello de botella técnico	Media	Carlos cubre TODO frontend → Jhulyam no toca UI. Emi cubre backend simple. Standups cada 4h para detectar bloqueos temprano.
Equipo pierde sincronía sin verse	Media	Standup de 5 min cada 4h (hora 4, 8, 12, 16, 20...)
Burnout en hora 30+	Alta	Turnos de sueño obligatorios (2 personas duermen 4h mientras otras 2 trabajan)
Scope creep ("agreguemos X feature")	Muy Alta	Regla: después de hora 16, congelar features. Solo pulir.

9. Plan B: pivot a Churn Hunters

"Construir un sistema que identifique clientes en riesgo de abandono antes de que ocurra y explique las razones detrás de esa predicción."

Trigger: Si en hora 16 el modo "ejecutar idéntico" NO funciona end-to-end → pivot.

Qué se reúsa de las primeras 16h de #4

- Next.js scaffold ☒
- Supabase setup + auth ☒
- Claude SDK config ☒
- UI shells de Carlos (dashboard layout adaptable) ☒
- Mockups de Marita (adaptables) ☒
- Pitch deck structure de Emi ☒

Lo que se pierde (~10-12h de trabajo): agent loop, browser automation con Stagehand, lógica de generalización. **Acéptenlo y muévase rápido.**

El ángulo ganador: Claude como modelo Y explicador

NO construyan regresión logística + SHAP/LIME. Esa es la versión genérica que harán otros 5 equipos. Diferenciación = una sola llamada a Claude que devuelve score + razones + acción recomendada.

```
Input: perfil del tendero (features estructuradas + historial)
  ↓
Claude (1 llamada con prompt estructurado)
  ↓
Output JSON: { score: 0.0-1.0, nivel: "alto"|"medio"|"bajo",
               razones: [...], acción_recomendada: "..."}

```

MVP de Churn Hunters en 24-30h restantes

Concepto: Dashboard que monitorea tenderos de Tuali y predice abandono con explicación accionable generada por Claude.

Stack adicional: Recharts (gráficos), Tanstack Table (lista priorizada).

Componentes:

1. **Mock dataset:** 200-500 tenderos con features (días sin pedido, caída en frecuencia, ticket promedio vs histórico, devoluciones recientes, zona, estacionalidad, etc.). 20-30 casos curados manualmente con labels "alto/medio/bajo" para el demo.
2. **Endpoint /api/predict:** recibe `tendero_id`, llama Claude con prompt estructurado, devuelve `{ score, nivel, razones, acción }`. Cache de resultados para no repetir llamadas.
3. **Dashboard principal:** lista priorizada (rojo/amarillo/verde) con score + 1-line razón + filtros por zona/segmento.
4. **Vista de detalle:** perfil del tendero + explicación completa + chart de tendencia + botón "generar plan de retención".
5. **Bonus (si hay tiempo):** chatbot lateral donde el ejecutivo pregunta "¿por qué este cliente está en riesgo?" y profundiza con Claude.

Killer demo (3 actos)

Acto 1 — El dolor (45 seg): "Arca tiene 100,000 tenderos. ¿Cómo saber cuáles están por irse antes de perderlos?" Mostrar lista plana de 500 tenderos sin priorización → caos.

Acto 2 — La inteligencia (60 seg): Cargar el dashboard con scoring de Claude. Lista ordenada por riesgo. Filtros funcionan en tiempo real.

Acto 3 — La explicación accionable (90 seg): Click en "José Pérez — 87% riesgo". Pantalla muestra:

"José tiene 87% probabilidad de abandono. Razones: (1) sin pedidos en 14 días (vs cada 7 días promedio histórico), (2) ticket promedio cayó 35% en últimos 2 meses, (3) competidor abrió tienda a 200m según

datos de zona. Acción recomendada: ejecutivo de cuenta llama mañana ofreciendo descuento del 8% en categoría Coca + visita de planeación de surtido."

Cierre (30 seg): "Cada tendero perdido = \$XXX,XXX de revenue anual. Si rescatamos el 30% de los flagged, son \$\$\$ millones recuperados. Y la explicación en lenguaje natural permite que cualquier ejecutivo actúe — no necesita ser data scientist."

10. Decisiones pendientes (resolver hoy)

- ☐ **Acceso a APIs/sandbox de Tuali:** preguntar a organizadores. Si no, mock portal propio (preferido).
- ☐ **Créditos de API:** pedir a organizadores créditos de Anthropic + Browserbase.
- ☐ **Lenguaje del agente:** TypeScript (todo en mismo repo) vs Python (mejor SDK de Anthropic) → **default: TypeScript** salvo objeción.
- ☐ **Roles finales:** ejecutar skill-check de 15 min al inicio.
- ☐ **Proceso específico a demostrar:** confirmar — sugerencia: "Actualizar surtido de SKUs por tienda en canal moderno".

11. Checklist hora 0 (los primeros 60 min)

- ☐ Todos en la misma sala / call permanente
- ☐ Skill-check de 15 min (ver sección 2)
- ☐ Confirmar roles finales
- ☐ Crear repo monorepo en GitHub
- ☐ Cada persona clona repo, instala Node 20 + pnpm + Python 3.11
- ☐ Crear cuentas: Anthropic Console, Browserbase, Supabase, Vercel, OpenAI (Whisper)
- ☐ Compartir API keys en `.env.example` y password manager seguro
- ☐ Jhulyam lee `skills/agent-harness-construction/SKILL.md`
- ☐ Carlos lee `skills/frontend-patterns/SKILL.md`
- ☐ Marita lee `skills/frontend-design-direction/SKILL.md`
- ☐ Emi lee `skills/frontend-slides/SKILL.md`
- ☐ Definir canal de comunicación rápido (Discord/Slack voice)
- ☐ Pedir créditos a organizadores
- ☐ Confirmar acceso a APIs Tuali (o decidir mock definitivamente)

Mantra del hackathon

Cada feature debe justificar su existencia con una pregunta: *¿esto se va a ver en el demo de 4 minutos?*

Si la respuesta es no, **NO se construye.**